

BÁO CÁO SẢN PHẨM DỰ THI

THIẾT KẾ VÀ TÍCH HỢP IP MÃ HÓA AES-256 VÀ SHA-256 CHO GIAO THỨC TRUYỀN VÀ NHẬN DỮ LIỆU

Nhóm dự thi: Hủ Tiếu

Đơn vị: Trường Đại học Công nghệ Thông tin và Truyền thông – Đại học Thái Nguyên

Liên hệ: huynhpham13790@gmail.com

TÓM TẮT SẢN PHẨM

Các thiết bị nhúng nhỏ thường truyền dữ liệu qua liên kết nối tiếp thô (UART, RS-485) vốn không có bảo mật ở tầng vật lý: ai chạm được vào đường dây đều đọc và sửa được từng byte. Sản phẩm là hai IP core mật mã tự thiết kế — AES-256 chế độ CTR và SHA-256 — cùng một hợp đồng giao diện dòng chảy chuẩn hóa (CSI v1.0), tích hợp vào một giao thức khung truyền/nhận chạy thật trên FPGA Sipeed Tang Nano 9K bằng toàn bộ toolchain mã nguồn mở. Điểm mới không nằm ở thuật toán mà ở lớp tích hợp: hai IP có cùng hình dạng cổng nên bộ định tuyến đối xử với chúng như nhau, đổi chỗ hai IP không cần sửa một dòng nào, và thêm IP thứ ba chỉ cần tăng một tham số. Kết quả đo trên phần cứng: 6664/8640 LUT4 (77%), F_{max} 46,85 MHz, 100 khung liên tiếp với 0% mất, độ trễ 32,42 ms và thông lượng 10 778 B/s — đạt 93,6% trần lý thuyết của UART 115200.

1. MỞ ĐẦU VÀ BÀI TOÁN

Cảm biến công nghiệp, thiết bị đo từ xa và module IoT giá rẻ hầu hết vẫn dùng UART hoặc RS-485 để đưa dữ liệu về bộ thu thập. Đây là liên kết hoàn toàn trần: kẻ tấn công tiếp cận được đường dây sẽ đọc trọn nội dung, và sửa được từng byte mà bên nhận không có cách nào phát hiện.

Hai hướng khắc phục hiện có đều có nhược điểm rõ. Chạy mã hóa bằng phần mềm trên vi điều khiển thì tiêu tốn chu kỳ CPU vốn đã hạn hẹp, và khóa nằm trong bộ nhớ chương trình nên dễ bị trích xuất. Mua IP core mật mã thương mại thì đắt và là hộp đen — người dùng không kiểm chứng được thứ mình đang tin tưởng, điều đặc biệt khó chấp nhận với một khối mật mã.

Đề tài chọn hướng thứ ba: tự thiết kế IP mật mã ở mức RTL, kiểm chứng bằng vector chuẩn của NIST, và quan trọng hơn — chuẩn hóa giao diện để chúng thực sự là "IP tích hợp được" chứ không phải hai khối logic nối tay. Toàn bộ dùng toolchain mã nguồn mở (Yosys, nextpnr-himbaechel, Project Apicula) trên một FPGA phổ thông giá dưới 200 nghìn đồng, nên mọi bước đều tái lập được mà không cần license.

2. MỤC TIÊU NGHIÊN CỨU VÀ PHẠM VI

- Mục tiêu 1: Thiết kế IP AES-256 chế độ CTR và IP SHA-256 ở mức RTL, đạt 100% vector chuẩn FIPS 197, FIPS 180-4 và NIST SP 800-38A.
- Mục tiêu 2: Chuẩn hóa một hợp đồng giao diện dùng chung cho cả hai IP, sao cho lớp giao thức không cần biết IP nào là AES, IP nào là SHA.
- Mục tiêu 3: Đạt chỉ tiêu định lượng trên phần cứng thật — vừa trong 8640 LUT4, $F_{max} \geq 27$ MHz, 0% mất khung trên 100 khung liên tiếp, thông lượng ≥ 8000 B/s.
- Phạm vi: Bảo vệ tính bí mật và toàn vẹn của khung dữ liệu trên đường truyền nối tiếp giữa hai thiết bị đã tin cậy nhau. Khóa AES nạp cứng lúc tổng hợp — hệ thống KHÔNG chống được kẻ tấn công đọc được bitstream, và trường digest không có khóa nên KHÔNG chống được kẻ tấn công chủ động biết thuật toán. Hai giới hạn này được phân tích ở mục 5.4.

3. NGHIÊN CỨU LIÊN QUAN VÀ ĐIỂM MỚI

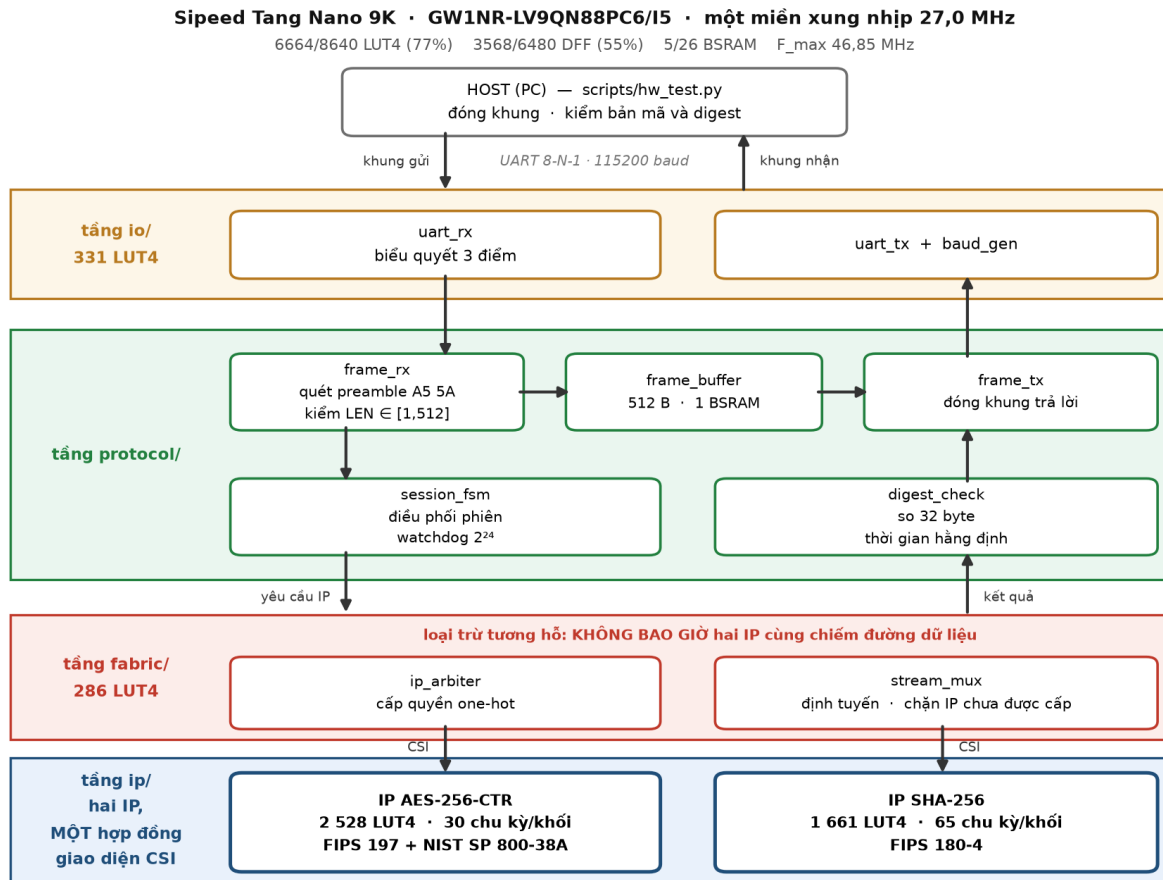
S-Box AES bằng số học trường composite $GF(((2^2)^2)^2)$ là hướng đã được Satoh và Canright đặt nền [3][4], cho diện tích nhỏ hơn nhiều so với bảng tra 256×8 . Các hiện thực AES/SHA trên FPGA nhỏ thường tối ưu từng lõi riêng lẻ và nối chúng bằng logic đặc thù.

Khoảng trống mà đề tài nhắm tới không phải ở lõi mật mã mà ở lớp tích hợp. Khi mỗi lõi có một kiểu giao diện riêng, lớp giao thức phải viết mã đặc thù cho từng lõi; thêm một thuật toán thứ ba đồng nghĩa sửa lại lớp giao thức. Đóng góp của đề tài gồm ba điểm:

- Hợp đồng giao diện CSI v1.0 — tập con tối giản kiểu AXI-Stream, đủ cho cả IP sinh dòng dữ liệu (AES) lẫn IP sinh kết quả thanh ghi (SHA), kèm 6 bất biến được kiểm tự động ở mọi chu kỳ mô phỏng.
- Tầng định tuyến cường chế loại trừ tương hỗ ở hai lớp độc lập, và phát hiện vi phạm ra đèn LED — biến một bất biến vốn chỉ kiểm được trong mô phỏng thành thứ quan sát được trên board.
- Chia sẻ tài nguyên có kiểm chứng: key schedule mượn 4 trong 8 khối S-Box của đường dữ liệu, tiết kiệm 324 LUT4 mà không ảnh hưởng tính đúng đắn.

4. GIẢI PHÁP VÀ KIẾN TRÚC PHẦN CỨNG

Thiết kế chia thành bốn tầng có ràng buộc phụ thuộc một chiều, được cường chế tự động bằng script trong bước lint: io/ (UART), protocol/ (ý nghĩa khung), fabric/ (cấp phát và định tuyến IP), ip/ (toán học mật mã thuần túy). Tầng ip/ không biết gì về ba tầng còn lại, nên hai IP dùng lại nguyên vẹn được ở dự án khác. Tầng protocol/ không được phép instantiate trực tiếp module trong ip/ — mọi truy cập phải đi qua fabric/. Chính hai quy tắc này biến sản phẩm từ "hai khối mật mã" thành "hai IP tích hợp được".



Hình 1. Kiến trúc bốn tầng, luồng dữ liệu một khung và số liệu tài nguyên đo được.

4.1. Cơ chế kỹ thuật

Một khung đi vào có dạng A5 5A || LEN(2) || IV(16) || PAYLOAD(LEN) || DIGEST(32), với DIGEST = SHA-256(LEN||IV||PAYLOAD). Bộ mở khung quét liên tục cặp byte preamble nên không giả định biên khung và tự phục hồi sau nhiễu; LEN được kiểm thuộc [1, 512] ngay khi đọc xong, trước khi cấp phát bộ đệm.

Máy trạng thái phiên xử lý theo năm bước: bầm LEN||IV||payload rồi so với DIGEST nhận được; nếu không khớp thì KHÔNG phát gì cả (fail closed); nếu khớp thì mã hóa payload tại chỗ bằng AES-256-CTR, bầm lại trên bản mã, rồi đóng khung trả lời. Phép so digest dùng XOR toàn bộ 256 bit rồi OR-reduce — thời gian hằng định, không có nhánh nào phụ thuộc dữ liệu.

Chế độ CTR được chọn có chủ đích: keystream_block = AES-Encrypt(counter + j) và ciphertext = plaintext XOR keystream. Vì XOR đối xứng nên cùng một khối cipher dùng cho cả hai chiều, và thiết kế KHÔNG cần hàm AES nghịch — không tồn tại InvSubBytes hay InvMixColumns trong toàn bộ mã nguồn, tiết kiệm khoảng một nửa diện tích so với hiện thực AES đầy đủ.

4.2. Kiến trúc RTL và giao diện

Hai IP có đúng cùng một tập cổng: nhóm điều khiển (csi_start / csi_mode / csi_busy / csi_done / csi_err), dòng dữ liệu vào và ra theo bắt tay valid/ready, và một cổng kết quả dạng thanh ghi. IP nào không dùng một nhóm thì buộc dây cố định chứ không được xóa cổng: SHA-256 đặt dòng ra bằng 0, AES-CTR đặt kết quả thanh ghi bằng 0.

Sáu bất biến của hợp đồng (done không lên khi chưa busy; done rộng đúng một chu kỳ; ready chỉ lên khi busy; busy phải lên trong hai chu kỳ sau start; err thì kết quả không hợp lệ; last xuất hiện đúng một lần mỗi thao tác) được một module kiểm tra chạy song song với IP trong mọi testbench, đếm vi phạm ở từng chu kỳ. Kiểm này đã bắt được lỗi thật: bản đầu của IP SHA-256 báo done ở nhánh lỗi mà chưa từng vào trạng thái busy.

AES-256 dùng kiến trúc lặp với 8 khối S-Box, hai chu kỳ mỗi vòng, tổng 30 chu kỳ mỗi khối 128 bit. Con số 8 là kết quả của một lần tối ưu có đo: bản 16 S-Box chiếm 2961 LUT4, vượt ngân sách, và phép đo cho thấy S-Box chiếm 44% diện tích IP nên đó mới là chỗ đáng cắt. SHA-256 dùng cửa sổ trượt 16 word thay vì lưu đủ 64 word của lịch trình thông điệp, tiết kiệm 1536 thanh ghi. Khóa vòng AES nằm trong bốn khối BSRAM 16×32 bit; đây cũng là một phát hiện phải trả giá bằng một lần P&R thất bại, trình bày ở mục 5.3.

5. THỰC NGHIỆM, KIỂM CHỨNG VÀ KẾT QUẢ

5.1. Thiết lập thực nghiệm

Nền tảng và công cụ:

FPGA / board	<i>Sipeed Tang Nano 9K — GW1NR-LV9QN88PC6/I5 (8640 LUT4, 6480 DFF, 26 BSRAM)</i>
Toolchain	<i>Yosys 0.68 + nextpnr-himbaechel 0.11.1 + Project Apicula — toàn bộ mã nguồn mở, không dùng license thương mại</i>
Clock / constraints	<i>Một miền xung nhịp duy nhất 27,0 MHz (create_clock -period 37.037). Không CDC, không FIFO bất đồng bộ</i>
Kiểm chứng chức năng	<i>6 testbench Icarus Verilog (S-Box, UART, SHA, AES, fabric, toàn hệ) với vector FIPS 197 / FIPS 180-4 / SP 800-38A</i>
Kiểm chứng phần cứng	<i>scripts/hw_test.py qua UART /dev/ttyUSB1; giá trị kỳ vọng sinh từ hiện thực tham chiếu Python (hashlib, cryptography)</i>

5.2. Kiểm chứng chức năng

Cả 6 testbench mô phỏng đều PASS. IP AES-256 vượt 17/17 phép kiểm, gồm vector khối đơn của FIPS 197 phụ lục C.3 và toàn bộ 4 khối liên tiếp của NIST SP 800-38A §F.5.5 — phép kiểm sau chứng minh bộ đếm CTR tăng đúng qua ranh giới khối, không chỉ đúng ở một khối đơn lẻ. IP SHA-256 vượt 17/17, gồm các độ dài biên 55, 56, 63, 64, 119 và 120 byte; hai giá trị 55 và 56 nằm hai bên ranh giới nơi phân đệm phải tràn sang khối thứ hai. S-Box được đối chiếu đủ cả 256 giá trị ở mức module con.

Trên phần cứng thật, chương trình kiểm quét 19 độ dài payload từ 1 đến 512 byte: mọi khung trả về đều có bản mã AES-256-CTR và digest SHA-256 khớp chính xác hiện thực tham chiếu, và giải mã lại cho đúng bản rõ ban đầu. LEN = 0 và LEN = 1024 đều bị loại đúng như đặc tả.

Ba kịch bản đối kháng — sửa một bit trong payload, cắt khung giữa chừng, gửi 32 byte rác trước preamble — đều cho kết quả đúng, và mỗi kịch bản đều chạy HAI PHA: pha A kiểm hệ thống im lặng, pha B gửi ngay một khung hợp lệ để chứng minh FPGA còn sống. Đây không phải hình thức: một bộ kiểm chỉ có pha A sẽ báo PASS ngay cả khi thiết kế đã chết hoàn toàn, và trong quá trình phát triển đã có đúng một lỗi làm toàn hệ câm lặng với triệu chứng bên ngoài y hệt hành vi từ chối đúng.

5.3. Kết quả triển khai phần cứng

Kết quả sau P&R trên GW1NR-LV9QN88PC6/I5:

Khối	LUT4	DFF	F_max	Chu kỳ/khối	Thông lượng	BSRAM
IP AES-256-CTR	2 528	1 357	—	30	14,4 MB/s	4
IP SHA-256	1 661	1 146	—	65	6,6 MB/s	0
protocol/ + fabric/ + io/	≈ 2 475	≈ 1 065	—	—	—	1
THIS WORK (toàn hệ)	6 664 / 8 640 (77%)	3 568 / 6 480 (55%)	46,85 MHz	—	10 778 B/s	5 / 26

Đo hiệu năng trên board với 100 khung liên tiếp, payload 128 byte: tỉ lệ mất khung 0,00%; độ trễ khứ hồi min 32,34 ms, trung bình 32,42 ms, tối đa 32,55 ms, độ lệch chuẩn 0,03 ms; thông lượng 10 778 B/s tương đương 86,2 kbps.

Độ lệch chuẩn 0,03 ms trên 100 khung cho thấy đường xử lý là tắt định — không có bộ đệm nào tràn và không có đường nào phụ thuộc dữ liệu. Thông lượng đạt 93,6% trần lý thuyết của UART 115200 (11 520 B/s); phần thiếu là 20 byte header và 32 byte digest của mỗi khung chứ không phải do engine chậm. Bản thân engine AES chạy ở 14,4 MB/s, tức nhanh hơn đường truyền hơn 1000 lần, nên nút thắt nằm hoàn toàn ở lớp vật lý.

Một kết quả đáng ghi lại: lần P&R đầu tiên THẤT BẠI dù báo cáo cùng lúc cho thấy LUT4 mới dùng 80% và DFF 60%. Nguyên nhân là bộ nhớ khóa vòng AES được khai báo dạng mảng 15×128 bit, trong khi cổng BSRAM của Gowin rộng tối đa 32 bit; công cụ tổng hợp rơi về RAM phân tán và sinh 32 khối RAM16SDP4, loại tài nguyên chỉ đặt được ở một số vị trí nhất định. Tách thành bốn bộ nhớ 16×32 bit đưa chúng vào BSRAM và P&R chạy qua ngay. Bài học: "còn 20% tài nguyên" không đồng nghĩa với "đặt chỗ được" — loại tài nguyên và ràng buộc vị trí quan trọng ngang số lượng.

5.4. Đánh giá mức độ bảo mật

Mô hình đe dọa: kẻ tấn công tiếp cận được đường dây nối tiếp giữa hai thiết bị, đọc và chèn được byte tùy ý, nhưng không truy cập được vào bên trong FPGA.

Những gì hệ thống bảo vệ được:

- Tính bí mật của payload. Dữ liệu đi trên dây ở dạng bản mã AES-256-CTR. Không có khóa thì không khôi phục được nội dung.
- Toàn vẹn trước nhiễu và sửa đổi ngẫu nhiên. Một bit sai bất kỳ trong payload hay digest đều làm khung bị loại; đã kiểm chứng trên phần cứng.
- Rò rỉ qua thời gian phản hồi. Phép so digest quét đủ 32 byte, không thoát sớm, nên không lộ vị trí byte sai. Độ lệch chuẩn độ trễ 0,03 ms trên 100 khung là bằng chứng định lượng cho tính tắt định này.

Những gì hệ thống KHÔNG bảo vệ được — công bố đầy đủ:

- Kẻ tấn công CHỦ ĐỘNG. Trường 32 byte là SHA-256(LEN||IV||PT) — một checksum toàn vẹn KHÔNG CÓ KHÓA, không phải MAC. Kẻ tấn công biết thuật toán có thể sửa payload rồi tính lại digest và bên nhận sẽ chấp nhận. Cách đúng là HMAC-SHA-256 (RFC 2104); đã hiện thực và vượt mô phỏng nhưng không vừa thiết bị (đo được 85–107% LUT4). Chúng tôi chọn nộp một thiết kế chạy thật kèm công bố giới hạn, thay vì một thiết kế đúng lý thuyết mà không nạp được lên board.
- Khóa nạp cứng. Không có cơ chế trao đổi khóa vì số học trường lớn (ECDH/RSA) vượt xa 8640 LUT4. Ai đọc được bitstream sẽ lấy được khóa.
- Tấn công kênh kề. Thời gian chạy AES là hằng định do kiến trúc lập không phụ thuộc dữ liệu, nhưng không có biện pháp chống phân tích công suất (DPA); che mặt nạ (masking) sẽ nhân đôi diện tích.

6. MỨC ĐỘ HOÀN THIỆN, DEMO VÀ KẾT LUẬN

6.1. Hồ sơ sản phẩm và video demo

MÃ NGUỒN	<i>github.com/felixsuhimitsu/cast-256 — nhánh feat/aes-sha256-ip-integration.</i> <i>Gồm 23 file RTL, 6 testbench, bộ kiểm phần cứng, và lớp tài liệu 11 bước trong docs/ (scope, spec, module map, kiến trúc, hợp đồng giao diện, WBS, ước lượng, rủi ro, gate, kế hoạch test, ma trận truy vết, 8 quyết định kiến trúc và nhật ký phát triển).</i>
VIDEO DEMO / QR	
TRẠNG THÁI HOÀN THIỆN	<i>[RTL ✓] [Simulation ✓ — 6/6 testbench PASS] [FPGA ✓ — chạy thật, 0% mất khung] [ASIC synthesis ✗ — ngoài phạm vi]</i>

6.2. Kết luận và khả năng ứng dụng

Đề tài đã hoàn thành hai IP mật mã tự thiết kế và chứng minh chúng tích hợp được vào một giao thức truyền/nhận chạy thật, bằng toàn bộ công cụ mã nguồn mở trên FPGA phổ thông. Ba kết quả định lượng chính: 6664/8640 LUT4 (77%) với $F_{\max}$ 46,85 MHz — dư 73% biên tần số so với yêu cầu 27 MHz; 100 khung liên tiếp với 0% mất và độ lệch chuẩn độ trễ 0,03 ms; và 19/19 độ dài payload từ 1 đến 512 byte cho bản mã cùng digest khớp chính xác hiện thực tham chiếu.

Mức độ hoàn thiện: RTL và mô phỏng đầy đủ, đã chạy trên phần cứng thật với bằng chứng log lưu kèm. Sản phẩm dùng được ngay cho các liên kết cảm biến–bộ thu thập trong mạng nội bộ đã tin cậy nhau. Hướng phát triển tiếp theo, theo thứ tự ưu tiên: thay digest không khóa bằng HMAC-SHA-256 bằng cách chia sẻ đường dữ liệu SHA giữa hai lần gọi thay vì dùng hai instance; bổ sung trao đổi khóa khi chuyển sang thiết bị lớn hơn; và cắm thêm một IP thứ ba (ChaCha20) để kiểm chứng lần nữa rằng hợp đồng giao diện thực sự mở.

TÀI LIỆU THAM KHẢO

- [1] NIST, FIPS PUB 197: Advanced Encryption Standard (AES), 2001.
- [2] NIST, FIPS PUB 180-4: Secure Hash Standard (SHS), 2015.
- [3] NIST, SP 800-38A: Recommendation for Block Cipher Modes of Operation, 2001.
- [4] D. Canright, "A Very Compact S-Box for AES", CHES 2005, LNCS 3659, pp. 441–455.
- [5] A. Satoh et al., "A Compact Rijndael Hardware Architecture with S-Box Optimization", ASIACRYPT 2001, LNCS 2248.
- [6] H. Krawczyk et al., RFC 2104: HMAC — Keyed-Hashing for Message Authentication, 1997.
- [7] YosysHQ, Yosys Open SYnthesis Suite và nextpnr, tài liệu chính thức, 2024.
- [8] Project Apicula — bitstream documentation cho FPGA Gowin, 2024.